

Sequential Pattern Mining

Outline

- Sequence database and sequential pattern mining
 - GSP algorithm
 - PrefixSpan algorithm
- The redundancy problem
 - Maximal and Closed patterns
- Constraint-based sequential pattern mining

Sequence Databases

- A sequence database consists of ordered elements or events
 - An element can consist of multiple items!
- Transaction databases vs. sequence databases

A transaction database

TID	itemsets
10	{a, b, d}
20	{a, c, d}
30	{a, d, e}
40	{b, e, f}

A sequence database

SID	Sequences
10	<a (abc) (ac) d (cf)>
20	<(ad) c (bc) (ae)>
30	<(ef) (ab) (df) c b>
40	<e g (af) c b c>

Itemset vs Sequence

- Itemsets are unordered, sequences are ordered
 - Itemset {a,b,c}: a, b and c occurred *together*
 - Sequence <a b c>: a was *followed* by b, which was *followed* by c
 - Sequence <a (bc)>: a was *followed* by b and c, which occurred *together*
- An itemset can contain no duplicate items, a sequence can
 - Sequence of elements, each element is an itemset
 - An element cannot contain duplicate items
 - <a (ab) (bc) c> is possible
 - <a (aa) (bb) c> is not

Applications

- Applications of sequential pattern mining
 - Customer shopping sequences:
 - First buy a computer, then an external hard disk, and then a digital camera, within 3 months.
 - Phonecall patterns, Weblog click streams
 - DNA sequences and gene structures
 - Medical treatments, natural disasters (e.g., earthquakes), science & eng. processes, stocks and markets, etc.

Subsequence vs. supersequence

- A sequence is an ordered list of elements (consisting of one or more items), denoted $\langle e_1 e_2 \dots e_l \rangle$
- Given two sequences $\alpha = \langle a_1 a_2 \dots a_n \rangle$ and $\beta = \langle b_1 b_2 \dots b_m \rangle$
- α is called a **subsequence** of β , denoted as $\alpha \subseteq \beta$, if there exist integers $1 \leq j_1 < j_2 < \dots < j_n \leq m$ such that $a_1 \subseteq b_{j_1}, a_2 \subseteq b_{j_2}, \dots, a_n \subseteq b_{j_n}$
- β is a **supersequence** of α
- $\langle (\text{ab}) \text{ d} \rangle$ is a subsequence of $\langle (\text{abc}) \text{ a} (\text{de}) \text{ f} \rangle$
- $\langle \text{a b} \rangle$ is not a subsequence of $\langle (\text{ab}) \rangle$
- $\langle (\text{ab}) \rangle$ is not a subsequence of $\langle \text{a b} \rangle$

More Terminology

- The **size** of a sequence is the number of elements (or itemsets) in the sequence
- The **length** of a sequence is the number of items in the sequence
 - A sequence of length k is called **k -sequence**
- The size of sequence $\langle a (bc) d \rangle$ is 3, and the length of this sequence is 4

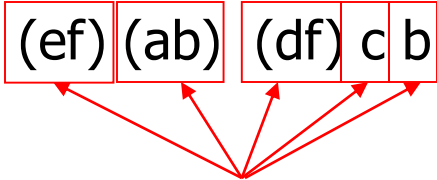
What Is Sequential Pattern Mining?

- Given a set of sequences and a support threshold, find the complete set of *frequent* subsequences

A *sequence database*

SID	sequence
10	<a (abc) (ac) d (cf)>
20	<(ad) c (bc) (ae)>
30	<(ef) (ab) (df) c b>
40	<e g (af) c b c>

A *sequence*: <(ef) (ab) (df) c b>



An element may contain a set of items. Items within an element are unordered and we list them alphabetically.

<a (bc) d c> is a *subsequence* of <a (ab) (ac) d (cf)>

Given *support threshold* $min_sup = 50\%$, <(ab) c> is a *(frequent) sequential pattern*

Challenges of Sequential Pattern Mining

- A **huge** number of possible sequential patterns are hidden in databases
- A mining algorithm should
 - find the **complete set of patterns**, when possible, satisfying the minimum support (frequency) threshold
 - be highly **efficient, scalable**, involving only a small number of database scans
 - be able to incorporate various kinds of **user-specific constraints**

Methods for sequential pattern mining

- **Apriori-based Approaches**
 - GSP
 - SPADE
 - etc.
- **Pattern-Growth-based Approaches**
 - FreeSpan
 - PrefixSpan
 - etc.
- **We will discuss GSP and PrefixSpan**

The Apriori Property of Sequential Patterns

- A basic property: Apriori (Agrawal & Srikant '94)
 - If a sequence S is not frequent, then none of the supersequences of S are frequent
 - e.g. $\langle h \ b \rangle$ is infrequent $\rightarrow$ so are $\langle h \ a \ b \rangle$ and $\langle (ah) \ b \rangle$

Seq. ID	Sequence
10	$\langle (bd)cb(ac) \rangle$
20	$\langle (bf)(ce)b(fg) \rangle$
30	$\langle (ah)(bf)abf \rangle$
40	$\langle (be)(ce)d \rangle$
50	$\langle a(bd)bcb(ade) \rangle$

Given support threshold
 $min_sup = 40\%$

GSP—Generalized Sequential Pattern Mining

- GSP (Generalized Sequential Pattern) mining algorithm
- Outline of the method
 - Initially, every item in DB is a candidate of length-1
 - for each level (i.e., sequences of length- k) do
 - scan database to collect support count for each candidate sequence
 - generate candidate length- $(k+1)$ sequences from length- k frequent sequences using Apriori
 - repeat until no frequent sequence or no candidate can be found
- Major strength: Candidate pruning by Apriori

Finding Length-1 Sequential Patterns

- Initial candidates:
 - $\langle a \rangle, \langle b \rangle, \langle c \rangle, \langle d \rangle, \langle e \rangle, \langle f \rangle, \langle g \rangle, \langle h \rangle$
- Scan database once, count support for candidates

$\text{min_sup} = 40\%$

Seq. ID	Sequence
10	$\langle (bd)cb(ac) \rangle$
20	$\langle (bf)(ce)b(fg) \rangle$
30	$\langle (ah)(bf)abf \rangle$
40	$\langle (be)(ce)d \rangle$
50	$\langle a(bd)bcb(ade) \rangle$

Cand	Sup
$\langle a \rangle$	0.6
$\langle b \rangle$	1.0
$\langle c \rangle$	0.8
$\langle d \rangle$	0.6
$\langle e \rangle$	0.6
$\langle f \rangle$	0.4
$\langle g \rangle$	0.2
$\langle h \rangle$	0.2

Generating Length-2 Candidates

51 length-2
Candidates

	<a>		<c>	<d>	<e>	<f>
<a>	<aa>	<ab>	<ac>	<ad>	<ae>	<af>
	<ba>	<bb>	<bc>	<bd>	<be>	<bf>
<c>	<ca>	<cb>	<cc>	<cd>	<ce>	<cf>
<d>	<da>	<db>	<dc>	<dd>	<de>	<df>
<e>	<ea>	<eb>	<ec>	<ed>	<ee>	<ef>
<f>	<fa>	<fb>	<fc>	<fd>	<fe>	<ff>

	<a>		<c>	<d>	<e>	<f>
<a>		<(ab)>	<(ac)>	<(ad)>	<(ae)>	<(af)>
			<(bc)>	<(bd)>	<(be)>	<(bf)>
<c>				<(cd)>	<(ce)>	<(cf)>
<d>					<(de)>	<(df)>
<e>						<(ef)>
<f>						

Without Apriori
property,
 $8*8 + 8*7/2 = 92$
candidates

Apriori prunes
44.57% candidates

Finding Length-2 Sequential Patterns

- Scan database one more time, collect support count for each length-2 candidate
- There are 19 length-2 candidates which pass the minimum support threshold
 - They are length-2 sequential patterns

The GSP Mining Process

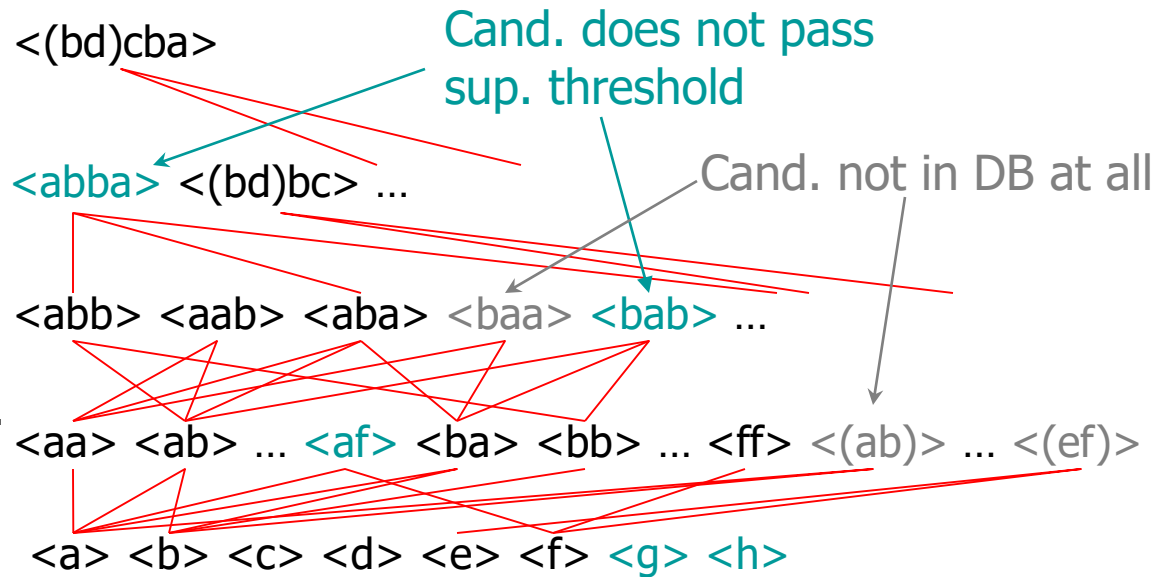
5th scan: 1 cand. 1 length-5 seq.
pat.

4th scan: 8 cand. 6 length-4 seq.
pat.

3rd scan: 46 cand. 19 length-3 seq.
pat. 20 cand. not in DB at all

2nd scan: 51 cand. 19 length-2 seq.
pat. 10 cand. not in DB at all

1st scan: 8 cand. 6 length-1 seq.
pat.



$min_sup = 40\%$

Seq. ID	Sequence
10	<(bd)cb(ac)>
20	<(bf)(ce)b(fg)>
30	<(ah)(bf)abf>
40	<(be)(ce)d>
50	<a(bd)bcb(ade)>

The GSP Algorithm

Algorithm GSP(S)

```
1   $C_1 \leftarrow \text{init-pass}(S);$  // the first pass over  $S$ 
2   $F_1 \leftarrow \{\langle \{f\} \rangle \mid f \in C_1, f.\text{count}/n \geq \text{minsup}\};$  //  $n$  is the number of sequences in  $S$ 
3  for ( $k = 2; F_{k-1} \neq \emptyset; k++$ ) do // subsequent passes over  $S$ 
4       $C_k \leftarrow \text{candidate-gen-SPM}(F_{k-1});$ 
5      for each data sequence  $s \in S$  do // scan the data once
6          for each candidate  $c \in C_k$  do
7              if  $c$  is contained in  $s$  then
8                   $c.\text{count}++;$  // increment the support count
9              end
10         end
11          $F_k \leftarrow \{c \in C_k \mid c.\text{count}/n \geq \text{minsup}\}$ 
12     end
13     return  $\bigcup_k F_k;$ 
```

Candidate Generation

Function candidate-gen-SPM(F_{k-1})

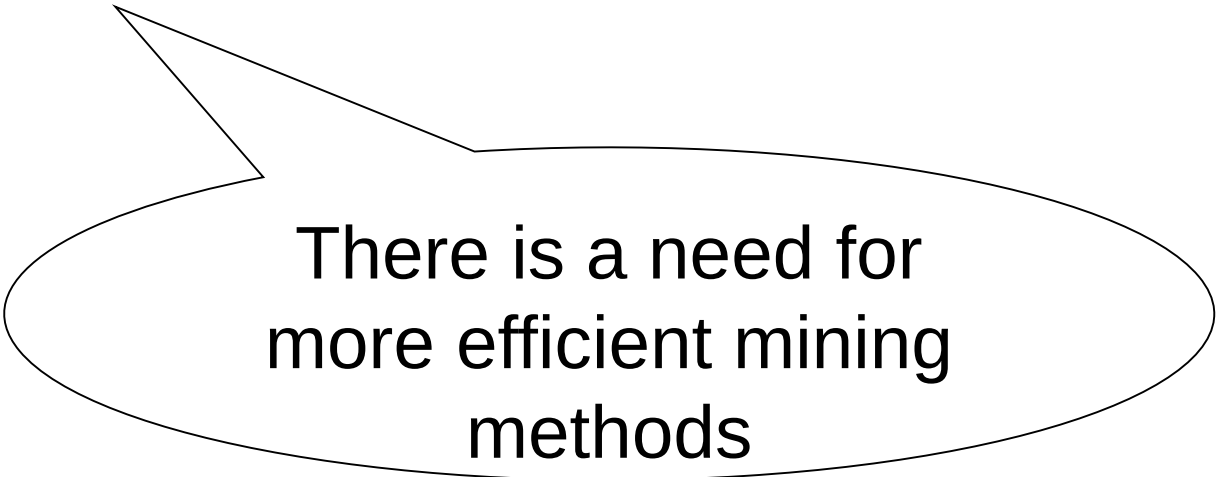
1. **Join step.** Candidate sequences are generated by joining F_{k-1} with F_{k-1} . A sequence s_1 joins with s_2 if the subsequence obtained by dropping the first item of s_1 is the same as the subsequence obtained by dropping the last item of s_2 . The candidate sequence generated by joining s_1 with s_2 is the sequence s_1 extended with the last item in s_2 . There are two cases:
 - the added item forms a separate element if it was a separate element in s_2 , and is appended at the end of s_1 in the merged sequence, and
 - the added item is part of the last element of s_1 in the merged sequence otherwise.When joining F_1 with F_1 , we need to add the item in s_2 both as part of an itemset and as a separate element. That is, joining $\langle \{x\} \rangle$ with $\langle \{y\} \rangle$ gives us both $\langle \{x, y\} \rangle$ and $\langle \{x\} \{y\} \rangle$. Note that x and y in $\{x, y\}$ are ordered.
2. **Prune step.** A candidate sequence is pruned if any one of its $(k-1)$ -subsequence is infrequent (without minimum support).

Details

- Lexicographical order is important
 - Each candidate can only be generated once, in a unique way
- Dropping the first item of s_1 and the last item of s_2 eliminates ambiguity
 - If we tried to combine $\langle ab \rangle$ and $\langle ac \rangle$, we would not know whether to generate $\langle abc \rangle$, $\langle acb \rangle$ or $\langle a(bc) \rangle$
 - Instead:
 - $\langle abc \rangle$ is generated from $\langle ab \rangle$ and $\langle bc \rangle$
 - $\langle acb \rangle$ is generated from $\langle ac \rangle$ and $\langle cb \rangle$
 - $\langle a(bc) \rangle$ is generated from $\langle ab \rangle$ and $\langle (bc) \rangle$
 - Ambiguity remains at level 2
 - $\langle ab \rangle$ and $\langle (ab) \rangle$ generated from $\langle a \rangle$ and $\langle b \rangle$

The GSP Algorithm

- Benefits from the Apriori pruning
 - Reduces search space
- Bottlenecks
 - Scans the database multiple times
 - Generates a huge set of candidate sequences



There is a need for
more efficient mining
methods

PrefixSpan (Prefix-Projected Sequential Pattern Growth)

- PrefixSpan
 - Projection-based
 - But only prefix-based projection: less projections and quickly shrinking sequences
- J.Pei, J.Han,... PrefixSpan : Mining sequential patterns efficiently by prefix-projected pattern growth. ICDE'01.
- (similar to FP-growth for itemsets)

Prefix and Suffix (Projection)

- $\langle a \rangle$, $\langle aa \rangle$, $\langle a(ab) \rangle$ and $\langle a(abc) \rangle$, etc. are prefixes of sequence $\langle a(abc)(ac)d(cf) \rangle$
- In fact, *any* subsequence can serve as a prefix!
- Suffix = what follows the first occurrence of the prefix
 - Underscore “_” used to indicate that the last element of the prefix was part of the first element of the suffix
- Given sequence $\langle a(abc)(ac)d(cf) \rangle$

Prefix	<u>Suffix</u> (Prefix-Based <u>Projection</u>)
$\langle a \rangle$	$\langle (abc)(ac)d(cf) \rangle$
$\langle aa \rangle$	$\langle (_bc)(ac)d(cf) \rangle$
$\langle ab \rangle$	$\langle (_c)(ac)d(cf) \rangle$

Mining Sequential Patterns by Prefix Projections

- Step 1: find length-1 sequential patterns
 - $\langle a \rangle$, $\langle b \rangle$, $\langle c \rangle$, $\langle d \rangle$, $\langle e \rangle$, $\langle f \rangle$
- Step 2: divide search space. The complete set of seq. pat. can be partitioned into 6 subsets:
 - The ones having prefix $\langle a \rangle$;
 - The ones having prefix $\langle b \rangle$;
 - ...
 - The ones having prefix $\langle f \rangle$

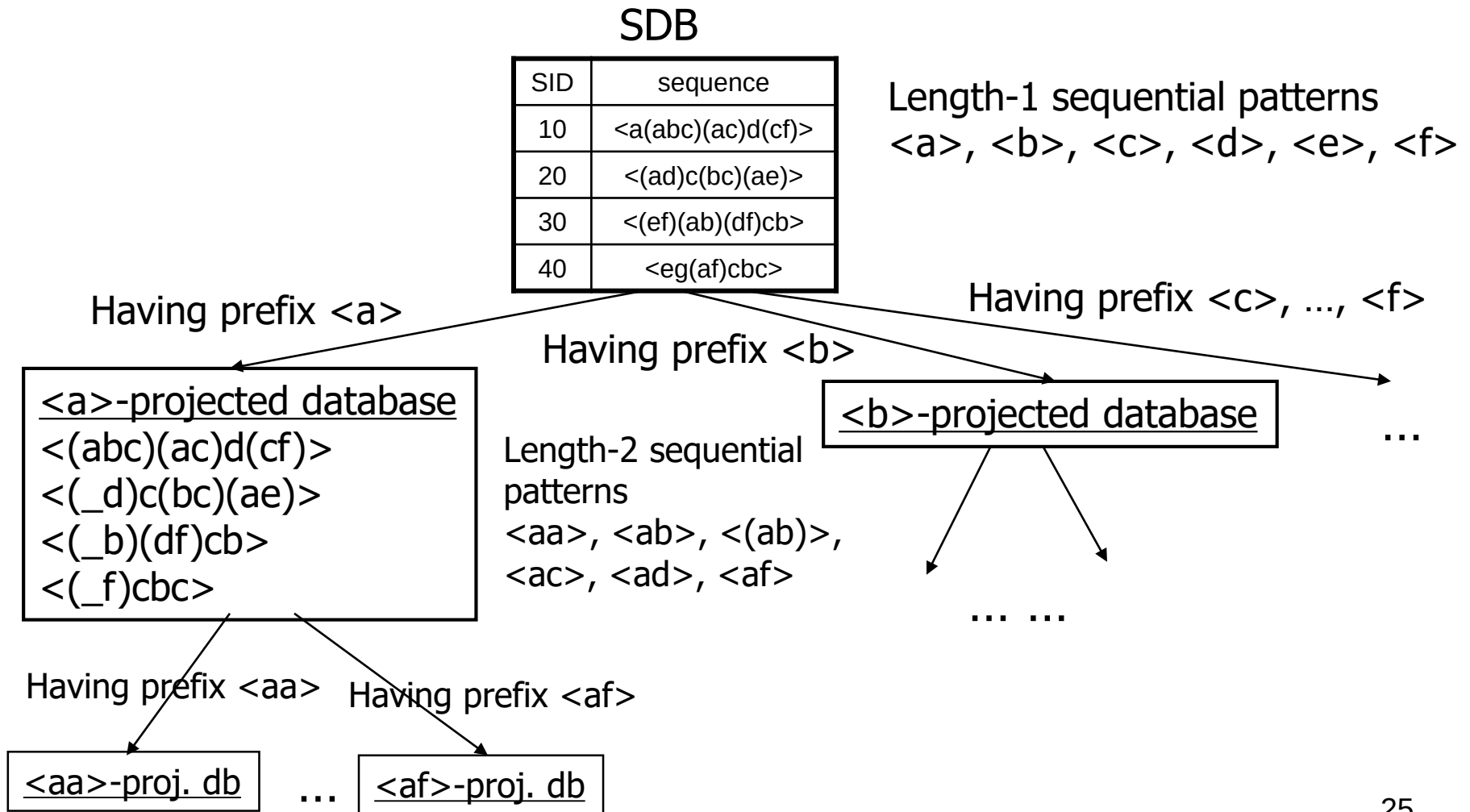
SID	sequence
10	$\langle a(abc)(ac)d(cf) \rangle$
20	$\langle (ad)c(bc)(ae) \rangle$
30	$\langle (ef)(ab)(df)cb \rangle$
40	$\langle eg(af)cbc \rangle$

Finding Seq. Patterns with Prefix <a>

- Only need to consider projections w.r.t. <a>
 - <a>-projected database: <(abc)(ac)d(cf)>, <(_d)c(bc)(ae)>, <(_b)(df)cb>, <(_f)cbc>
- Find all the length-2 seq. pat. having prefix <a>:
<aa>, <ab>, <(ab)>, <ac>, <ad>, <af>
 - Further partition into 6 subsets
 - Having prefix <aa>;
 - ...
 - Having prefix <af>

SID	sequence
10	<a(abc)(ac)d(cf)>
20	<(ad)c(bc)(ae)>
30	<(ef)(ab)(df)cb>
40	<eg(af)cbc>

Completeness of PrefixSpan



The Algorithm of PrefixSpan

- **Input:** A sequence database S , and the minimum support threshold min_sup
- **Output:** The complete set of sequential patterns
- **Method:** Call $\text{PrefixSpan}(<>, 0, S)$
- **Subroutine** $\text{PrefixSpan}(\alpha, l, S|\alpha)$
- **Parameters:**
 - α : sequential pattern,
 - l : the length of α ;
 - $S|\alpha$: the α -projected database, if $\alpha \neq <>$; otherwise the sequence database S

The Algorithm of PrefixSpan(2)

- **Method**

1. Scan $S|\alpha$ once, find the set of frequent items b such that:
 - a) b can be added to the last element of α to form a sequential pattern; or
 - b) $\langle b \rangle$ can be added as a new element to α to form a sequential pattern.
2. For each frequent item b , append it to α (as determined above) to form a sequential pattern α' , and output α' ;
3. For each α' , construct α' -projected database $S|\alpha'$, and call PrefixSpan(α' , $l+1$, $S|\alpha'$).

Efficiency of PrefixSpan

- No candidate sequence needs to be generated
 - Only sequences that are actually present in the data are evaluated
- Projected databases keep shrinking
 - Sequences get shorter
 - Number of sequences decreases
- Major cost of PrefixSpan: constructing projected databases

Apriori-based v Projection-based

- GSP (et al.) v PrefixSpan (et al.)
 - GSP uses breadth-first-search (BFS)
 - PrefixSpan uses depth-first search (DFS)
- Output is the same
- Runtime and memory usage varies
 - Dense data tends to favour GSP-like algorithms
 - Sparse data tends to favour PrefixSpan-like algorithms
 - Most data is sparse

The Redundancy Problem

- The result set often contains many similar sequences
- Example:
 - Assume $\langle a \ (bc) \ d \rangle$ is frequent
 - Then the following sequences also MUST be frequent:
 $\langle a \rangle, \langle b \rangle, \langle c \rangle, \langle d \rangle$
 $\langle a \ b \rangle, \langle a \ c \rangle, \langle a \ d \rangle, \langle b \ d \rangle, \langle c \ d \rangle, \langle (bc) \rangle,$
 $\langle a \ (bc) \rangle, \langle a \ b \ d \rangle, \langle a \ c \ d \rangle, \langle (bc) \ d \rangle$
- Outputting all these frequent subsequences carries little additional information!

Maximal Patterns

- A frequent sequence s is **maximal** if no supersequence of s is frequent
 - i.e., $s \subset s' \Rightarrow s'$ is infrequent
- Output significantly smaller than frequent sequential patterns
- The set of all frequent sequences can be reconstructed from the maximal sequences
- However, the support of all frequent sequences cannot be derived from the maximal sequences
 - e.g., if $\text{sup}(\langle abc \rangle) = 10 \geq \text{minsup}$, then $\text{sup}(\langle ab \rangle) \geq \text{minsup}$, but $\text{sup}(\langle ab \rangle) = ?$

Closed Patterns

- A frequent sequence s is **closed** if no supersequence of s has the same support
 - i.e., $s \subset s' \Rightarrow \text{support}(s') < \text{support}(s)$
- Output typically a lot smaller than frequent sequential patterns
- The set of all frequent sequences can be reconstructed from the closed sequences
- The support of all frequent sequences can also be derived from the closed sequences
 - The support of a non-closed sequence is equal to the support of its most frequent supersequence

Maximal and Closed Patterns

- Example with minsup = 40%:

- $\text{sup}(\langle ac \rangle) = 60\%$
- $\text{sup}(\langle acd \rangle) = 60\%$
- $\text{sup}(\langle acde \rangle) = 40\%$
- $\text{sup}(\langle acdef \rangle) = 20\%$

- Frequent patterns:

- $\langle ac \rangle, \langle acd \rangle, \langle acde \rangle$

- Closed patterns:

- $\langle acd \rangle, \langle acde \rangle$

- Maximal patterns:

- $\langle acde \rangle$

Maximal Patterns are Closed

- Proof:

Assume s is a maximal sequential pattern

$\Rightarrow s$ is frequent and for each s' such that $s \subset s'$, s' is infrequent

$\Rightarrow s$ is frequent and for each s' such that $s \subset s'$, $\text{support}(s') < \text{support}(s)$

$\Rightarrow s$ is a closed sequential pattern

- Maximal patterns $\subset$ Closed patterns $\subset$ Frequent patterns
- These concepts also exist for other pattern types, such as itemsets

Algorithms

- Closed:

- CloSpan: PrefixTree with additional pruning [Yan et al. 2003]
- BIDE: Memory-efficient forward/backward checking [Wang&Han 2007]
- ClaSP: Based on SPaDE [Gomariz et al. 2013]

– Maximal:

- AprioriAdjust: Based on Apriori [Lu & Li 2004]
- VMSP: Based on vertical data structures [Fournier-Viger et al. 2014]

Constraints for Seq.-Pattern Mining

- Item constraint
 - Find web log patterns only about online-bookstores
- Length constraint
 - Find patterns having at least 20 items
- Superpattern constraint
 - Find super patterns of “PC → digital camera”
- Aggregate constraint
 - Find patterns such that the average price of items is over \$100

More Constraints

- Regular expression constraint
 - Find patterns “starting from the homepage, search for hotels in Washington DC area”
 - `homepage(WashingtonDC|DC)(hotel|motel|lodging)`
- Duration constraint
 - Find patterns such that the gap between the first and the last item is 24 hours
- Gap constraint
 - Find purchasing patterns such that “the gap between consecutive purchases is less than 1 month”

Summary

- Sequential Pattern Mining is useful in many applications, e.g., weblog analysis, financial market prediction, BioInformatics, etc.
- It is similar to frequent itemsets mining, but gives importance to the order of the items.
- We have looked at different approaches that are descendants from two popular algorithms in mining frequent itemsets
 - Candidates Generation: GSP, SPADE, ...
 - Pattern Growth: FreeSpan, PrefixSpan, ...